

T17 — Gestione degli Errori ed Architettura delle Eccezioni

Gerarchia Throwable, Checked vs Unchecked, try-catch-finally, Try-with-resources e AutoCloseable

Docente: Prof. Michele Pasqua — **Appunti Revisione:** Wally

A.A. 2025/2026 – Università degli Studi di Verona

0.1 1. Analisi Teorica Approfondita (Slide T17)

La lezione 17 affronta la gestione degli errori e delle anomalie a runtime attraverso il meccanismo delle **Eccezioni** in Java.

0.1.1 1.1 Limiti della Gestione Tradizionale degli Errori (Slide 3-6)

Nei linguaggi procedurali privi di eccezioni (come C) o nell'uso ingenuo dei metodi, gli errori vengono segnalati tramite **valori sentinella** (es. restituire -1, null, Float.MAX_VALUE):

```
1 float division(int num, int den) {
2     if (den != 0)
3         return (float) num / den;
4     else
5         return Float.MAX_VALUE; // Valore speciale di errore
6 }
```

Perché questo approccio è inadeguato?

1. **Inquinamento della logica applicativa:** Il chiamante deve ricordarsi di controllare continuamente i valori di ritorno con una ragnatela di if-else.
 2. **Ambiguità semantica:** A volte il valore sentinella può essere un risultato matematico o di dominio assolutamente lecito.
 3. **Inapplicabilità ai Costruttori:** Un costruttore non ha tipo di ritorno (void implicito); se i parametri sono non validi, non può restituire -1 per segnalare il fallimento!
 4. **Difficoltà di propagazione:** Se l'errore avviene a 10 livelli di profondità nello stack di chiamate, ogni singola funzione intermedia dovrebbe propagare manualmente il codice d'errore fino al punto in cui può essere gestito.
-

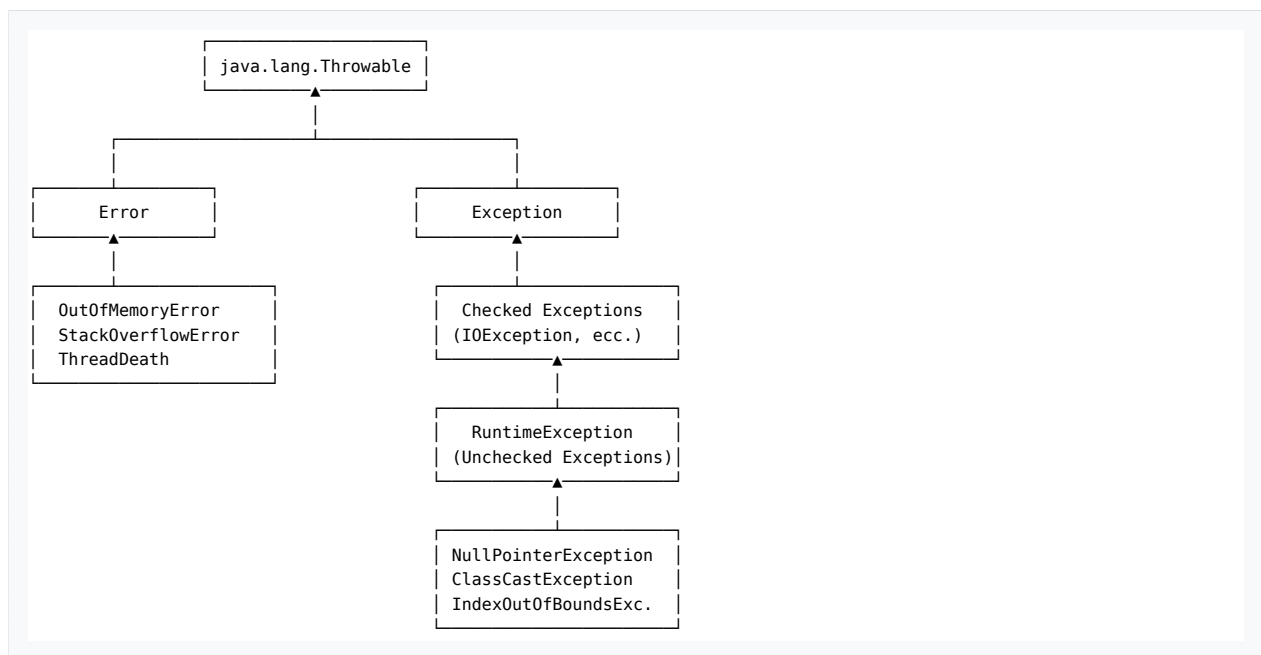
0.1.2 1.2 La Filosofia delle Eccezioni in Java: try-catch-throw (Slide 7-10)

Java disaccoppia la **logica di business** (*cosa deve fare il programma quando tutto va bene*) dalla **logica di gestione degli errori** (*cosa fare quando accade un'anomalia*):

- * **try**: racchiude il blocco di codice a rischio di eccezione.
- * **throw**: solleva attivamente un'istanza di eccezione nel punto in cui l'anomalia viene riscontrata.
- * **catch**: cattura l'eccezione ed esegue il codice di ripristino/notifica.
- * **Interruzione immediata**: quando viene eseguito throw, il metodo **interrompe immediatamente la propria esecuzione**, svuota lo stack frame corrente e risale lungo la catena di chiamate (*call stack*) fino al primo blocco catch compatibile.

0.1.3 1.3 La Gerarchia delle Eccezioni in Java (Slide 20-21)

Tutto l'albero discende da `java.lang.Throwable`:



1. Error (Unchecked):

- Condizioni anomale e catastrofiche interne alla JVM o al sistema operativo (`OutOfMemoryError`, `StackOverflowError`).
- Il codice utente non deve tentare di catturarle, poiché lo stato della memoria della JVM è compromesso.

2. Exception:

- Anomalie derivanti da condizioni esterne o logiche. Si ramificano in due grandi famiglie:
 - **Checked Exceptions** (sottoclassi dirette di `Exception`, escluse le `RuntimeException`).
 - **Unchecked Exceptions** (sottoclassi di `RuntimeException`).

0.1.4 1.4 Checked vs Unchecked Exceptions (Slide 22-24)

Caratteristica	Checked Exceptions (extends Exception)	Unchecked Exceptions (extends RuntimeException)
Natura dell'errore	Situazioni anomale prevedibili ma esterne (es. FileNotFoundException, rete disconnessa).	Errori del programmatore (bug logici, precondizioni violate, puntatori nulli).
Controllo del compilatore	Obbligatorio (<i>Catch or Declare rule</i>). Il codice non compila se non gestite con try-catch o dichiarate con throws.	Opzionale . Il compilatore non richiede alcuna dichiarazione o cattura.
Firma del metodo	Modificata: public void foo() throws MyCheckedException	Invariata: nessun throws richiesto.
Diffusione (<i>Propagazione</i>)	<i>“Si propagano come un virus”</i> (Slide 24): costringono tutti i chiamanti intermedi a dichiarare throws o gestire.	Risalgono naturalmente lo stack fino al main o al gestore globale.
Raccomandazione moderna (Slide 24)	Definirle solo se l'utente del metodo può ragionevolmente intraprendere un'azione di recupero .	Preferite per errori di validazione dello stato o degli argomenti.

0.1.5 1.5 “Exception Dirty Tricks”: Trasformare Checked in Unchecked (Slide 25-26)

Un pattern comune quando una checked exception sporcherebbe decine di firme senza che il chiamante intermedio possa gestirla:

```

1 public void bar() { // Nessuna dichiarazione throws richiesta!
2     try {
3         foo(); // foo() dichiara throws MyCheckedException
4     } catch (MyCheckedException e) {
5         throw new RuntimeException(e); // Incapsulata e rilanciata come Unchecked!
6     }
7 }

```

L'eccezione originale non viene persa: è memorizzata come causa (*chained exception* visibile nel messaggio Caused by: dello stacktrace).

0.1.6 1.6 Eccezioni nei Cicli (*Exceptions and Loops*, Slide 27-28)

- **Errore sulla singola iterazione (Slide 27):** il blocco try-catch è **all'interno** del ciclo. Se l'iterazione fallisce, viene intercettata e il ciclo prosegue con la successiva (es. lettura da input utente fino a valore corretto).
- **Errore fatale per l'intero ciclo (Slide 28):** il blocco try-catch **avvolge** il ciclo. Se si verifica un'anomalia, il ciclo viene interrotto definitivamente e l'esecuzione salta direttamente al catch esterno.

0.2 2. Analisi Dettagliata del Codice (Lezione16/MavenDate)

In Lezione16/MavenDate il prof. Pasqua applica i principi della Slide T17 rimuovendo i vecchi System.out.println("Illegal date!") e introducendo la gestione strutturata degli errori tramite il package it.oop.exception.

0.2.1 2.1 Le Nuove Eccezioni: `IllegalDateException.java` e `OrderdPairException.java`

1. `IllegalDateException.java` (Unchecked)

```
1 package it.oop.exception;
2
3 public class IllegalDateException extends RuntimeException {
4     public IllegalDateException(String message) {
5         super(message);
6     }
7 }
```

- Estende `RuntimeException`: è una **Unchecked Exception**.
- Modella una violazione delle precondizioni sui valori numerici (giorno, mese, anno).
- Non costringe tutti i client a dichiarare `throws IllegalDateException`.

2. `OrderdPairException.java` (Checked)

```
1 package it.oop.exception;
2
3 public class OrderdPairException extends Exception { // Notare il typo del docente "Orderd"
4     public OrderdPairException(String message) {
5         super(message);
6     }
7 }
```

- Estende direttamente `Exception`: è una **Checked Exception**.
- **Impatto immediato sulla firma:**

– In `OrderedPair.java`:

```
1 public OrderedPair(T first, T second) throws OrderdPairException {
2     this.first = first;
3     this.second = second;
4     if (first.compareTo(second) > 0)
5         throw new OrderdPairException("Not orderd pair");
6 }
```

– In `DateInterval.java`:

```
1 public DateInterval(Date left, Date right) throws OrderdPairException {
2     super(left, right); // Invoca super() che lancia la checked exception, quindi
                          // DateInterval DEVE dichiarare throws!
3 }
```

0.2.2 2.2 Refactoring del Metodo `verify()` in `Date.java`

```
1 void verify() {
2     if (year < 0)
3         throw new IllegalDateException("Illegal date: wrong year");
4     if (month < 1 || month > 12)
5         throw new IllegalDateException("Illegal date: wrong month");
6     if (day < 1 || day > daysPerMonth(month))
7         throw new IllegalDateException("Illegal date: wrong day");
8 }
```

- **Miglioramento architetturale:**
 - Prima: un giorno errato (es. 35) stampava un messaggio sulla console ma creava comunque un oggetto `Date` incoerente nell'Heap.

T17 — Gestione degli Errori ed Architettura delle Eccezioni

- Ora: il costruttore viene interrotto da `throw` prima che l'assegnazione sia completata. **È impossibile creare un'istanza con stato non valido!**

0.2.3 2.3 MainDate.java e l'Applicazione di Tutti i Pattern della Slide T17

```

1 package it.oop.ui;
2
3 import it.oop.core.*;
4 import it.oop.core.Date;
5 import it.oop.exception.IllegalDateException;
6 import it.oop.exception.OrderdPairException;
7
8 import java.util.InputMismatchException;
9 import java.util.Scanner;
10
11 public class MainDate {
12
13     public static void main(String[] args) {
14         System.out.println("Insert day, month, year as numbers");
15         int d = -1, m = -1, y = -1;
16         boolean correct = false;
17
18         // 1. Pattern Slide 27: try-catch dentro il ciclo per recuperare l'errore di input
19         // dell'utente
20         while (correct == false) {
21             try {
22                 Scanner sc = new Scanner(System.in);
23                 d = sc.nextInt();
24                 m = sc.nextInt();
25                 y = sc.nextInt();
26                 correct = true; // Se uno dei nextInt() fallisce, questa riga non viene raggiunta
27             } catch (InputMismatchException ime) {
28                 System.out.println("Input not valid, retry");
29             }
30
31             // 2. Cattura della Unchecked Exception IllegalDateException per notifica pulita
32             try {
33                 FormattedDate date = new ItalianDate(d, m, y);
34                 System.out.println(date.toString());
35             } catch (IllegalDateException ide) {
36                 System.out.println("Date not valid: " + ide.getMessage());
37             }
38
39             // 3. Pattern Slide 26 ("Dirty Trick"): incapsulamento di Checked Exception in
40             // RuntimeException
41             try {
42                 DateInterval di = new DateInterval(new Date(31, 1, 2025), new Date(1, 1, 2025));
43             } catch (OrderdPairException ope) {
44                 ope.printStackTrace();
45                 throw new RuntimeException(ope); // Rilanciata come unchecked!
46             }
47 }

```

0.3 3. Risultato di Esecuzione Reale da Terminale

T17 — Gestione degli Errori ed Architettura delle Eccezioni

0.3.1 Test 1: Input con data errata (35 1 2025)

Esecuzione:

```
1 mvn exec:java -Dexec.mainClass="it.oop.ui.MainDate" <<< "35 1 2025"
```

Output:

```
1 Insert day, month, year as numbers
2 Date not valid: Illegal date: wrong day
3 it.oop.exception.OrderdPairException: Not orderd pair
4     at it.oop.core.OrderedPair.<init>(OrderedPair.java:13)
5     at it.oop.core.DateInterval.<init>(DateInterval.java:7)
6     at it.oop.ui.MainDate.main(MainDate.java:35)
7 ...
8 Caused by: it.oop.exception.OrderdPairException: Not orderd pair
```

0.3.2 Analisi dell'output:

1. Date not valid: Illegal date: wrong day: il costruttore di ItalianDate delega a Date.verify(), che lancia IllegalDateException("Illegal date: wrong day"), catturata dal blocco catch (IllegalDateException ide) che stampa il messaggio di errore controllato.
2. DateInterval di = new DateInterval(31/1/2025, 1/1/2025): poiché la data iniziale 31/1/2025 è cronologicamente successiva alla data finale 1/1/2025 (violazione dell'invariante start <= end), OrderedPair solleva la checked OrderdPairException. Il blocco catch stampa lo stack trace con ope.printStackTrace() e poi rilancia l'errore incapsulato con throw new RuntimeException(ope);, terminando l'esecuzione con codice di uscita d'errore (build failure) come previsto.

Dimmi “**vai**” per procedere con il **Blocco 17** (Lezione 18 / Slide T18 — *Streams and Functional Programming*, Stream API, operazioni intermedie vs terminali, map, filter, reduce, collect e il nuovo file MainStream.java in Lezione17/MavenDate).